

ETIS

Master Manual & Documentation Index

Executive overview of the Engineering Studio, its place in ETIS, student and instructor use, installation, administration, production operations, CLI tooling, and recovery

Audience	Students, teaching staff, technical administrators, production operators, institutional adopters, and maintainers
Purpose	Provide the front door to the complete ETIS Engineering Studio documentation library and explain how the product, people, evidence, and operating model fit together.
Reference implementation	COMP 330/474 Software Engineering • Fall 2026
Version	Semester Start 2026 • August 23, 2026

CORE IDEA

ETIS Engineering Studio is an engineering apprenticeship environment. It makes evidence and engineering reasoning visible, gives students bounded senior-review-style challenge and coaching, and preserves the central rule that the student remains the responsible engineer. It is developmental; formal phase-gate authority remains with the instructor and the tagged repository evidence.

Contents

Executive Overview

1. Why ETIS Engineering Studio exists
2. Where Studio fits in ETIS
3. The Engineering Education Suite
4. Studio from the student perspective
5. Studio from the instructor and course-owner perspective
6. Semester administration and course operations
7. Installation, architecture, and technical administration
8. Production operations and the Azure Operations CLI
9. Backup, recovery, and disaster recovery
10. Security, privacy, authority, and evidence boundaries
11. Documentation library index
12. Scenario-based document finder
13. Institutional adoption and governance
14. Documentation lifecycle and publishing conventions

Appendix A - Terminology

Appendix B - Role and responsibility matrix

Appendix C - First-adoption checklist

HOW TO USE THIS MANUAL

Read the Executive Overview and Sections 1–3 for context. Then jump to the role or scenario you actually have. This manual explains the system and points to the detailed companion document that contains the procedure, screenshots, troubleshooting, and appendices.

Executive Overview

ETIS Engineering Studio is the developmental-review component of the ETIS Engineering Education Suite. It connects real project evidence in GitHub with structured, phase-aware engineering review conversations. The purpose is not to automate engineering judgment. The purpose is to help students practice it: recognizing evidence, surfacing uncertainty, defending tradeoffs, responding to findings, and making decisions that they can own.

The broader ETIS environment deliberately separates several responsibilities that are easy to blur in AI-assisted education. The Framework defines the discipline. The Engineering Platform provides practical stages, templates, examples, and guidance. ETIS Preflight checks readiness near a formal gate. Engineering Studio supports repeated developmental review. The Engineering Review Center supports the instructor's formal phase-gate review of tagged repository evidence. The project repository remains the durable evidence record.

ETIS ecosystem: discipline, guidance, education, evidence, and review

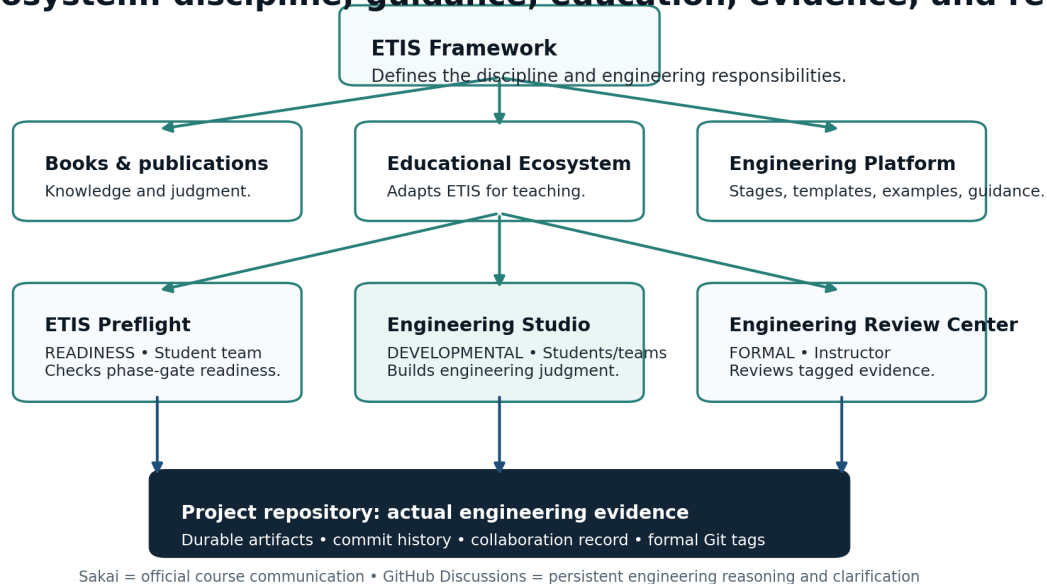


Figure 1. ETIS ecosystem and the role of Engineering Studio. Studio is connected to the Framework and Platform, but it has a distinct developmental-review purpose.

EXECUTIVE POSITION

Studio should be understood as part of an evidence-centered engineering learning system, not as a chatbot added to a course. Its value comes from the relationship among authentic repository evidence, controlled reviewer roles, longitudinal context, instructor governance, and production-grade operation.

1. Why ETIS Engineering Studio exists

Modern software teams can produce code quickly, including with generative AI. That makes engineering judgment more important, not less. Students still need to determine what should be built, what evidence is credible, who owns a decision, what risks remain, what should be verified, and what happens when a technical choice is wrong.

Educational challenge

Studio response

Educational challenge	Studio response
Students can produce artifacts without understanding them.	Reviewers ask what the evidence actually demonstrates and require the student to explain the engineering meaning.
AI can make an answer look authoritative even when evidence is weak.	Studio distinguishes observations from review interpretations, exposes gaps, and encourages disagreement when the reviewer missed evidence.
Traditional assignments can reward final artifacts while hiding reasoning.	Frozen evidence snapshots and individual review histories preserve the context in which a student reasoned.
Teams can mistake activity for evidence.	Studio evaluates repository artifacts, workflow signals, ownership, and phase-relevant evidence rather than rewarding commit counts.
Students may optimize for what they think the grader wants.	Studio is explicitly developmental. Formal phase-gate review is separate and grounded in tagged repository evidence.

- Practice asking better engineering questions, not merely producing answers.
- Make uncertainty explicit and learn when “I do not know yet” is the responsible position.
- Trace decisions to evidence, consequences, owners, and change triggers.
- Experience disagreement, challenge, and correction as normal professional engineering behavior.
- Build durable evidence in the repository so the work can stand independently of the conversation that helped improve it.

2. Where Studio fits in ETIS

ETIS is broader than a single course or application. COMP 330/474 uses a selective two-cycle implementation of the larger lifecycle. The six course phase gates are an educational implementation, not the complete ETIS lifecycle.

ETIS element	Primary purpose	What it is not
ETIS Framework	Defines the discipline: responsibilities, evidence, tradeoffs, governance, and lifecycle thinking.	Not a course-specific application.
Books & publications	Develop conceptual knowledge and engineering judgment.	Not the operational workflow.
Educational Ecosystem	Adapts ETIS for teaching models, durations, and outcomes.	Not the student evidence record.
Engineering Platform	Provides ES-100–ES-114 stages, templates, examples, governance assets, and practical guidance.	Not Preflight, Studio, or formal review.
ETIS Preflight	Checks whether phase-gate evidence appears ready for formal review.	Not the formal reviewer or grade authority.
Engineering Studio	Develops engineering judgment through repeated evidence-grounded developmental review.	Not a self-service public app or grading AI.

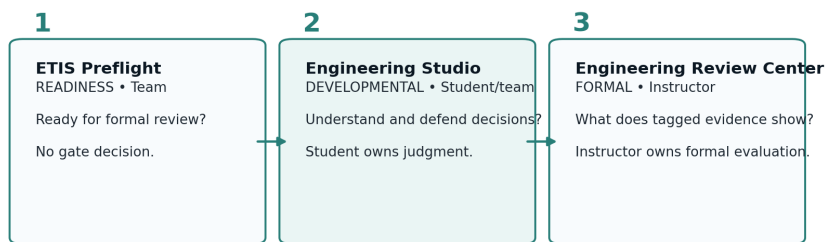
ETIS element	Primary purpose	What it is not
Engineering Review Center	Supports formal instructor-led phase-gate review of tagged evidence.	Does not replace instructor academic authority.
GitHub repository	Preserves actual engineering artifacts, commits, collaboration signals, and formal tags.	Not merely a code bucket; it is the durable evidence record.

COURSE COMMUNICATION

Sakai remains the authoritative course announcement and management environment. GitHub Discussions is for persistent shared engineering questions and reasoning. The deeper ETIS website/Platform remains the place for framework guidance, examples, templates, and technical reference.

3. The Engineering Education Suite

Engineering Education Suite: three different questions, three different authority boundaries



Iterative flow: build → Studio → improve → Studio → Preflight → improve → tag → Review Center

Studio is developmental. Formal review is grounded in the designated Git tag and repository evidence.

Figure 2. The three suite capabilities deliberately separate readiness, developmental review, and formal review.

The typical student workflow is iterative. Studio can be used many times during a phase; Preflight is most useful as the team approaches formal readiness; the Review Center is formal and uses the designated Git tag as the evidence boundary.

Student engineering loop: evidence improves through repeated judgment

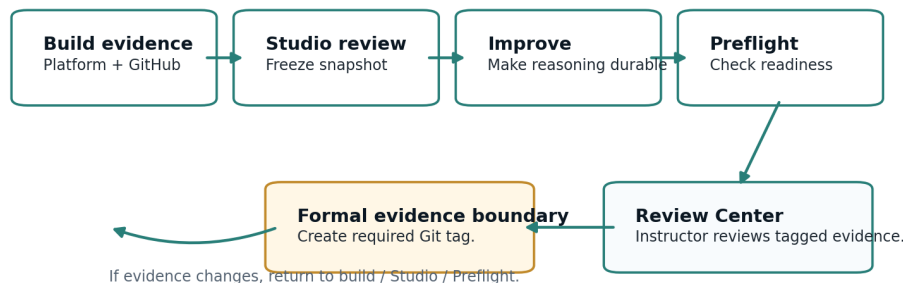


Figure 3. Typical student evidence-and-review loop. A formal tag is created only after the evidence is ready to stand on its own.

4. Studio from the student perspective

Students experience Studio as an apprenticeship environment. They sign in with institutional identity, establish their team and GitHub identity, connect one verified team repository, inspect phase-aware Engineering Evidence, and choose the review interaction that best fits the current need.

Student task	What happens	Detailed reference
Sign in and establish context	Institutional identity authenticates the student; roster/team authorization determines what the student may access.	05 Student User Guide
Link GitHub identity	Studio associates the student with the GitHub account used for engineering identity; repository evidence access remains a separate GitHub App boundary.	05, 06
Verify the team repository	The team nominates the actual working repository; GitHub owner/admin authorization and exact-repository verification establish trusted evidence.	06 Team GitHub & Repository Setup Quickstart
Inspect Engineering Evidence	Students see strengths, phase evidence, findings, professional lenses, and provenance.	05 Student User Guide
Choose a review	Board Review lets the board choose the most valuable phase question; Focused Review lets the student choose the engineering subject; Review Findings works directly with existing findings.	05, 07
Work in the Review Room	The evidence snapshot freezes automatically. The student can ask, disagree, seek a nudge, challenge evidence, and form a recommendation.	05 Student User Guide
Revisit history	Review History preserves individual reasoning against the snapshot used at that time. GitHub remains authoritative for repository commit history.	05 Student User Guide

STUDENT BOUNDARY

Team evidence and repository binding are shared; coaching conversations and learning state are individual. Starting a review freezes the evidence used for that session. New commits do not rewrite old reviews.

5. Studio from the instructor and course-owner perspective

Teaching staff use Studio to understand engineering evidence and student reasoning without reducing the course to activity counts. The Instructor Command Center surfaces teams that may need attention, Team Detail connects people, repository state, evidence, and active reviews, and persisted review conversations can be inspected by authorized teaching staff.

Instructor surface	Purpose	Boundary
--------------------	---------	----------

Instructor surface	Purpose	Boundary
Command Center	Class-level engineering intelligence, attention signals, active review context, AI economics.	Signals support judgment; they are not autonomous grades.
Teams / Team Detail	Inspect repository connection, team membership, evidence state, active reviews, and bounded recovery actions.	Do not rewrite student evidence or silently change trusted repository state.
Students	Add/reactivate/deactivate students and manage team membership while preserving history.	Role authorization controls who may mutate roster/team state.
Reviews	Inspect persisted student review turns and status.	Draft/unsent student text remains private; teaching staff do not impersonate students.
Engineering Evidence	Inspect FACT observations, REVIEW interpretations, provenance, disputes, and longitudinal change.	Findings are review interpretations, not immutable truth.
AI Usage & Cost	Operational intelligence for model usage, latency, token volume, and estimated cost.	Not a student evaluation surface.
Semester Setup	Create terms/sections, import roster, create teams, control releases, assign staff, archive.	Restricted to course-administration authority.

Detailed reference: 04 - Instructor & Course Owner Handbook. Administrative semester workflow: 10 - Semester Administration & Course Operations Guide.

6. Semester administration and course operations

The semester lifecycle is an authorization and records boundary, not just a calendar convenience. A term in setup is administrative preparation. An active term supports normal student/team/review activity. An archived term is historical and read-only; it cannot grant current authorization.

1. Create the course term once and add the required section(s).
2. Import the roster and intentionally decide how to handle students missing from later imports.
3. Create teams and assign students; later moves preserve membership history.
4. Review A1–A6 availability, due, accept-until, and release behavior by section. Sakai remains the deadline authority.
5. Assign teaching staff using the minimum role necessary.
6. Activate only after roster, team, identity, repository, schedule, and staff checks are complete.
7. Operate the semester through adds, drops, team moves, repository recovery, schedule changes, and evidence/review monitoring.
8. Archive after the term ends. Archiving stops operational authority while preserving historical reviews, evidence snapshots, membership, findings, and usage records.

DETAILED PROCEDURE

Use 10 - Semester Administration & Course Operations Guide for semester setup, activation, mid-semester changes, and archive. Use 04 for the day-to-day teaching-staff experience.

7. Installation, architecture, and technical administration

The reference deployment is production-engineered rather than a classroom-only prototype. The installation path spans institutional identity, GitHub integration, CI/CD, Azure infrastructure, PostgreSQL, Key Vault, DNS/TLS, observability, backup/recovery, and course provisioning.

Layer	Reference implementation	Primary document
Human identity	Microsoft Entra / Loyola SSO; authorization remains course/section scoped.	02 Installation & Administration
GitHub identity	OAuth links the student's GitHub identity; repository reading uses the GitHub App.	02, 03
Repository evidence access	Selected-repository GitHub App scope; exact repository verification; fail closed on broad scope.	02, 03
Application runtime	Azure Container Apps; revision mode Single; accepted min/max replica baseline 1 / 5.	02, 03, 08
Persistence	Azure Database for PostgreSQL Flexible Server with Alembic-managed schema lifecycle.	02, 03, 11
Secrets	Azure Key Vault / managed identity where applicable; deployment secrets in protected GitHub environment.	02, 03
Deployment	Protected branch → PR → CI → merge → GitHub Deploy Azure workflow → post-deployment acceptance.	02, 08, 09
Observability	Application Insights, Log Analytics, Azure Monitor alerts, CLI health/status/log/drift checks.	08, 09

A first-time installer should begin with 02 - Installation & Administration Guide and use 03 - Architecture & Detailed Design alongside it whenever a step crosses a trust, evidence, authorization, or failure boundary.

8. Production operations and the Azure Operations CLI

Production responsibility stack: install once, operate continuously, recover deliberately

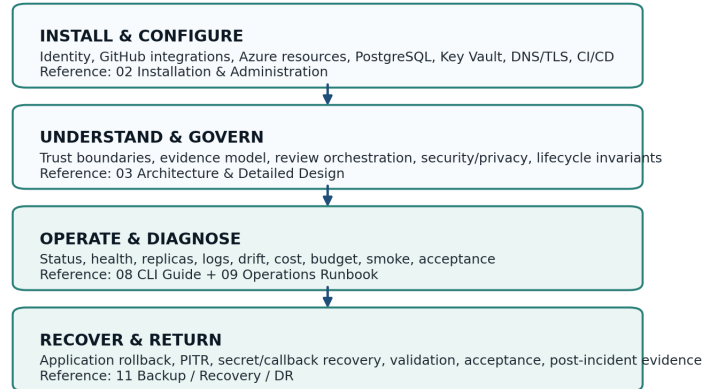


Figure 4. Production responsibility stack. Installation establishes the system; operations and recovery preserve it over time.

The operator-facing CLI is the preferred routine interface for production inspection and acceptance. It intentionally does not provide a deployment command. Deployment remains controlled through repository governance and the GitHub workflow.

Command	Question it answers
doctor	Is this workstation and Azure access ready for operations?
status	Is production in the expected operating state?
config	What important non-secret configuration is live?
health	Are public health/readiness endpoints healthy and responsive?
replicas	Are configured and running replicas healthy?
revisions	Which revisions exist and where is traffic directed?
logs	What meaningful production activity/errors are visible?
cost	What month-to-date cost telemetry is currently available?
budget	Is the accepted budget/alert configuration present?
smoke	Does the externally visible production path behave correctly?
drift	Does live runtime match source-of-truth and accepted invariants?
acceptance	Do all required post-deployment production checks pass?
scale	Apply a guarded temporary live replica override when deliberately required.

OPERATOR DEFAULT

Use 08 - Azure Operations CLI User Guide for command semantics and examples. Use 09 - Production Operations Runbook when starting from a symptom or alert. Cost telemetry is advisory and does not silently become a production-health gate.

9. Backup, recovery, and disaster recovery

Recovery is deliberately separated from ordinary troubleshooting. Operators should preserve evidence, identify the failure boundary, choose the least destructive recovery layer, validate data/schema/identity behavior, and run production acceptance before declaring service restored.

Recovery situation	Preferred response	Reference
Bad application deployment	Roll back to a known-good immutable application image/revision; validate migrations and external behavior.	11 DR Runbook
Database data/schema incident	Use PostgreSQL recovery/PITR decision flow; restore to a temporary server when appropriate; validate before cutover.	11 DR Runbook
Secret/Key Vault problem	Repair the secret/identity boundary without dumping values; restart/redeploy only as required.	11 DR Runbook
DNS/TLS/callback mismatch	Restore external alignment among custom domain, Entra callback, GitHub callback/setup URLs, and application origin.	02, 11
Production alert / uncertain symptom	Triage first with status, health, replicas, logs, smoke, and drift. Escalate to recovery only when justified.	09 Operations Runbook

RETURN-TO-SERVICE GATE

Recovery is not complete when a resource is merely “running.” Required validation culminates in the same post-deployment production acceptance discipline used after controlled deployment.

10. Security, privacy, authority, and evidence boundaries

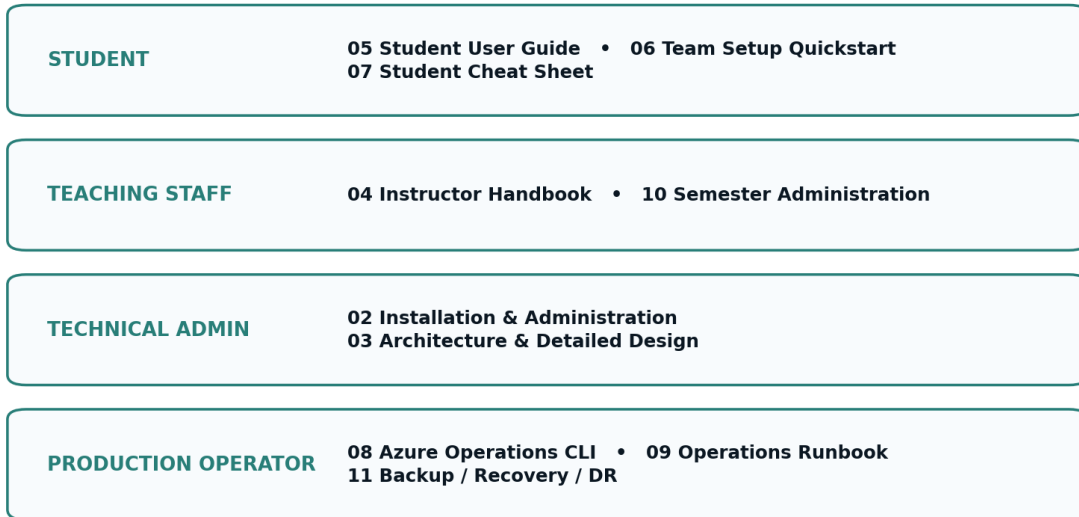
Several invariants are central to the product and should guide any future change, adoption, or troubleshooting decision.

- Authentication and authorization remain separate. A valid institutional identity does not automatically grant course/team access.
- GitHub identity linking and GitHub repository evidence access are separate trust boundaries.
- Repository access is selected-repository, read-oriented, and verified against the exact nominated repository.
- Evidence snapshots are immutable historical context for the review session that used them.
- FACT observations and REVIEW interpretations are distinct; findings can be challenged and corrected.
- Students remain responsible engineers. Controlled reviewers provide bounded advice, challenge, and coaching.
- Studio is developmental. Formal phase-gate review is based on the designated Git tag in the Engineering Review Center.

- Instructor/course-owner authority is role-scoped; archived terms are historical and cannot grant current operational authorization.
- Routine operational output avoids displaying secret values; recovery procedures should preserve the same discipline.

11. Documentation library index

Documentation architecture: choose by audience and task



00 Master Manual = executive context + audience map + scenario index + terminology + document governance

Figure 5. Documentation architecture by audience. This Master Manual is the front door to the complete library.

ID	Document	Primary audience	Use it when...
00	Master Manual & Documentation Index	All audiences	System purpose, ETIS relationships, audience map, scenario index, governance, and the front door to the library.
02	Installation & Administration Guide	Technical administrator / institutional adopter	Greenfield install, Entra, GitHub OAuth/App, GitHub production environment, Azure, DNS/TLS, deployment, acceptance, administration, upgrade, decommissioning.
03	Architecture & Detailed Design	Architect / maintainer / security reviewer	Trust boundaries, data/evidence model, review orchestration, identity, lifecycle, persistence, concurrency, Azure topology, invariants, extension points.
04	Instructor & Course Owner Handbook	Instructor / course owner / teaching staff	Command Center, teams, students, reviews, evidence, findings, repository recovery, AI usage, support, coaching boundary, troubleshooting.

ID	Document	Primary audience	Use it when...
05	Student User Guide	Student	Complete student experience: sign-in, team/repository trust, evidence, snapshots, reviews, recommendations, findings, history, help, troubleshooting.
06	Team GitHub & Repository Setup Quickstart	Student team	Fast, controlled team repository onboarding and GitHub authorization/verification troubleshooting.
07	Student Cheat Sheet / Quick Reference	Student	Compact review-choice, recommendation, findings, history, and troubleshooting reference for normal use.
08	Azure Operations CLI User Guide	Production operator / administrator	Every CLI command and option, healthy output, PASS/WARN/FAIL interpretation, examples, scale guardrails, post-deploy usage.
09	Production Operations Runbook	Production operator / incident lead	Symptom-first monitoring, incident triage, auth/GitHub/database/runtime/OpenAI/cost/drift cases, evidence capture, return to service.
10	Semester Administration & Course Operations Guide	Course owner / instructor	Term/section creation, roster, teams, A1–A6 release controls, teaching staff, activation, mid-semester changes, archive.
11	Backup, Recovery & Disaster Recovery Runbook	Technical administrator / recovery lead	Rollback, PostgreSQL PITR, schema/data validation, secret/callback recovery, disaster scenarios, DR exercises, recovery acceptance.

DOCUMENT 01

This Master Manual intentionally serves the executive-overview role for the published library. The earlier standalone executive overview is superseded by this document unless a separate short executive brief is later published for institutional leadership.

12. Scenario-based document finder

If you are trying to...	Start here	Then use
Understand what Studio is and why it exists	00 Master Manual	03 for deep architecture; 05/04 for role-specific use
Get a student/team connected for the first time	06 Team Setup Quickstart	05 Student User Guide for full context
Choose between Board, Focused, and Review Findings	07 Student Cheat Sheet	05 for complete examples and review behavior
Support a student who says Studio “got it wrong”	04 Instructor Handbook	05 for the student challenge/correction model

If you are trying to...	Start here	Then use
Prepare the semester before students arrive	10 Semester Administration	04 for daily teaching-staff use
Install a new institutional deployment	02 Installation & Administration	03 Architecture; 11 for recovery readiness
Understand why an installation step exists	03 Architecture & Detailed Design	02 for the exact procedure
Check production quickly	08 CLI Guide	09 if any command indicates a problem
Respond to an Azure Monitor alert or outage report	09 Operations Runbook	08 for exact command details; 11 if recovery is required
Restore the database or roll back after a serious failure	11 Backup/Recovery/DR	09 for incident evidence and return-to-service context
Audit privacy/security/authority boundaries	03 Architecture	02, 04, 10, 11 as needed by boundary

13. Institutional adoption and governance

COMP 330/474 is the flagship reference implementation, but adopting institutions may deploy and configure one or more ETIS educational capabilities for their own courses. Adoption transfers operational responsibility to the institution; it does not transfer that responsibility to students.

- Choose and operate institutional identity, authorization, GitHub organization/App ownership, privacy policy, retention policy, AI provider/configuration, monitoring, and support ownership.
- Define who may be Course Owner, Instructor, TA, Reviewer, technical administrator, and production operator.
- Preserve the distinction between developmental Studio activity and formal academic evaluation.
- Establish repository ownership and GitHub App authorization responsibilities before class starts.
- Operate backup/recovery, alerting, cost controls, and post-deployment acceptance as real production responsibilities.
- Adapt the course phase-gate model without implying that the COMP 330/474 six-gate sequence is the complete ETIS lifecycle.

14. Documentation lifecycle and publishing conventions

The documentation set is designed as a coordinated library. A product change is complete only when the affected operating, teaching, student, installation, architecture, and recovery documentation has been checked for impact.

Change type	Documents that normally require review
Student UI / review behavior	05, 06, 07; possibly 04 and 00
Instructor / semester workflow	04, 10; possibly 05 and 00
Identity / GitHub onboarding	02, 03, 04, 05, 06, 09, 11, 00

Change type	Documents that normally require review
Azure/IaC/deployment baseline	02, 03, 08, 09, 11, 00
CLI command behavior	08, 09, 11; possibly 02
Recovery / retention / archive semantics	03, 10, 11, 00
Product naming / ETIS suite terminology	All public-facing documents

- Use current names: ETIS Preflight, ETIS Engineering Studio, ETIS Engineering Review Center, Engineering Platform.
- Avoid “grading AI,” “AI grader,” generic “agents,” and older Instructor Engineering Workbench terminology in current-facing material.
- Keep screenshots tightly cropped to the instructional point and do not expose secret values.
- Treat the repository/IaC and deployed product as authoritative for technical defaults; update documents when the accepted production baseline changes.
- Maintain the shared ETIS visual system: dark navy header, teal accents, restrained callouts, task-oriented tables, and role-specific troubleshooting.

Appendix A - Terminology

Term	Meaning
ETIS	Engineering Trustworthy Intelligent Systems; the broader engineering framework and ecosystem.
Engineering Platform	Practical ETIS guidance environment with stages, templates, examples, governance and evidence-centered assets.
ETIS Preflight	Phase-gate readiness analysis used by student teams before formal review.
Engineering Studio	Developmental engineering apprenticeship environment using frozen repository evidence and controlled reviewers.
Engineering Review Center	Instructor-facing formal phase-gate review environment using tagged evidence as the formal boundary.
Evidence snapshot	Frozen repository evidence state used by a Studio review. Later commits do not rewrite the historical snapshot.
Formal evidence boundary	The designated Git tag establishing what is formally submitted for phase-gate review.
FACT	Evidence-supported observation about the repository/project state.
REVIEW	Interpretation, concern, or finding that can be challenged, corrected, resolved, accepted, or deferred.

Term	Meaning
Controlled engineering reviewers	Defined reviewer roles with bounded mandates; not generic open-ended chat agents.
CourseTerm status	setup = administrative preparation; active = normal semester; archived = historical/read-only and non-authorizing.

Appendix B - Role and responsibility matrix

Role	Primary responsibility	Typical documents
Student	Build evidence, use Studio developmentally, own decisions, use Preflight near readiness, create formal tag when instructed.	05, 06, 07
Team	Maintain shared repository evidence, ownership, process, and repository authorization.	05, 06
Instructor	Coach, inspect team/review/evidence signals, exercise academic judgment, support students.	04
Course Owner	Own semester setup, staff roles, section lifecycle, high-authority course mutations.	04, 10
Technical Administrator	Install/configure identity, GitHub integration, Azure, database, secrets, DNS/TLS, deployment.	02, 03
Production Operator	Monitor, diagnose, run acceptance, handle alerts, preserve incident evidence.	08, 09
Recovery Lead	Choose rollback/PITR/recovery strategy, validate restored service, preserve evidence.	11
Institution	Own deployment, privacy, retention, security, AI service configuration, support and operational governance.	00, 02, 03, 09, 10, 11

Appendix C - First-adoption checklist

1. Read this Master Manual and agree on product boundaries, terminology, and ownership.
2. Assign institutional technical administrator, Course Owner, production operator, and recovery lead responsibilities.
3. Read 03 Architecture & Detailed Design before finalizing identity/GitHub/privacy/retention choices.
4. Execute 02 Installation & Administration from a clean environment; do not improvise around trust boundaries.
5. Run post-deployment production acceptance and record the known-good baseline.
6. Prepare 10 Semester Administration before roster/team onboarding.
7. Give instructors 04 and students 05/06/07 before the first required use.
8. Establish routine monitoring and incident ownership using 08 and 09.

9. Perform or schedule a recovery exercise using 11 before the deployment becomes business-critical.
10. At semester end, archive the term and confirm historical records remain readable but non-operational.

PUBLISHING BASELINE

This manual and its companion documents are designed to be read together. The Master Manual explains why and where; each companion document explains exactly how for the relevant audience and scenario.